Passive Buzzer Instrument/Song Recorder

Connor Yip
cyip022
6/10/26

Introduction

This project is an instrument/recorder that can both play notes on command and record them for later playback. All of this is output on a buzzer which plays tones based on the direction of the joystick. The note being played, the most recent note, the current octave/pitch of the notes, the amount of notes recorded, and their octave are all displayed on a small LCD screen.

Elements of Complexity

ST7735 128x128 SPI LCD Screen

Displays the majority of information that the user needs

User Guide

The user plays a note by pointing the joystick in a given direction. Starting from directly down, and going in a full circle, every note between C4 and B4 can be played. While the user is tilting the joystick, the note is played through the buzzer, and the note being played flashes in the middle of the screen

When the user wants to change the octave of the note being played/recorded, the user will press the right button, where it will cycle between octaves starting at C4, C5, and C3. The LED will shine either blue, green, or red so they know what octave they're in

When the user wants to record a note, they point in the direction of the note they want to record, let go of the joystick, and press the left button briefly. A block at the top of the screen will be drawn, whose color will be that of the current octave, and the LED will flash cyan for a brief moment. No more than 32 notes can be recorded at a given time.

When the user wants to delete a note, they press down on the joystick. The right-most block in the top of the screen is cleared, and the LED flashes cyan briefly

When the user wants to play the recorded notes, they will hold down the left button for at least two seconds, and the song will play when released. Notes will play sequentially, and the LED will be white for as long as the song plays for.

Hardware Components Used

Input

- 2 push buttons
- 1 joystick potentiometer

Output

- 1 passive buzzer
- 1 RGB LED
- 1 ST7735 128x128 SPI LCD screen

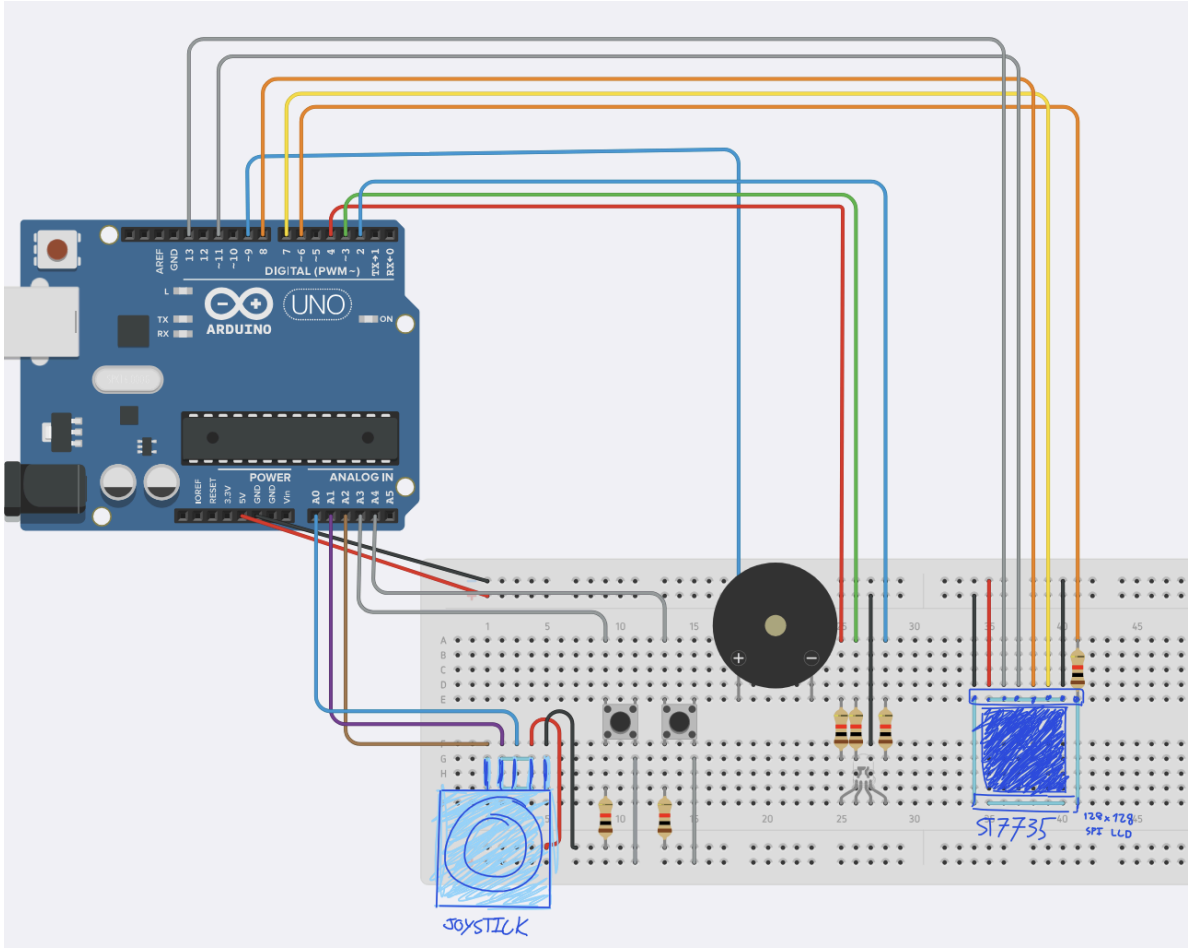
Misc.

- 6 220 ohm resistors

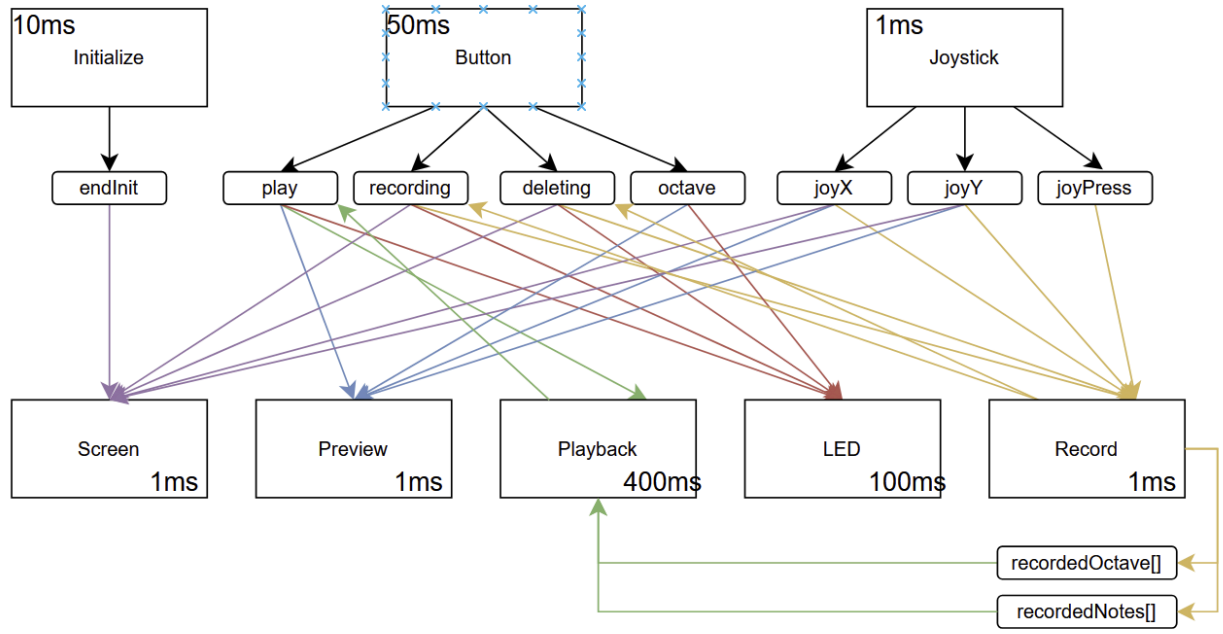
Software Libraries Used

The only library used was <math.h>, which was used to calculate the angle the joystick is pointed in.

Wiring Diagram



Task Diagram



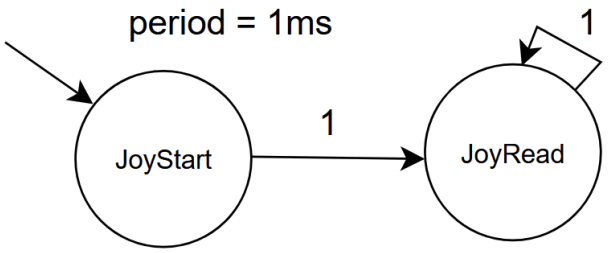

```
double joyX = 0;                //X direction of joystick
double joyY = 0;                //Y direction of joystick
unsigned char joyPress = 0;     //joystick Press
unsigned char left = 0;         //button variables
unsigned char right = 0;        //button variables
unsigned char endInit = 0;      //to tell Program that initialization ended
unsigned char octave = 1;       //affects buzzer tone
unsigned char play = 0;         //turns playback mode on/off;
unsigned char recording = 0;    //tells recording task to record a note
unsigned char deleting = 0;     //tells recoridng task to delete a note
unsigned char noteAmount = 0; //tracks number of notes recorded. never exceeds 32.
int notes[] = {523, 554, 587, //for playing/recording notes.
               622, 659, 698,
               740, 784, 831,
               880, 932, 988};
int recordedNotes[32] = {0};  //self explanatory
int recordedOctave[32] = {0}; //so that the right octave is played
```

SynchSM Diagrams

“*” represents abstracted action logic

Joystick

period = 1ms

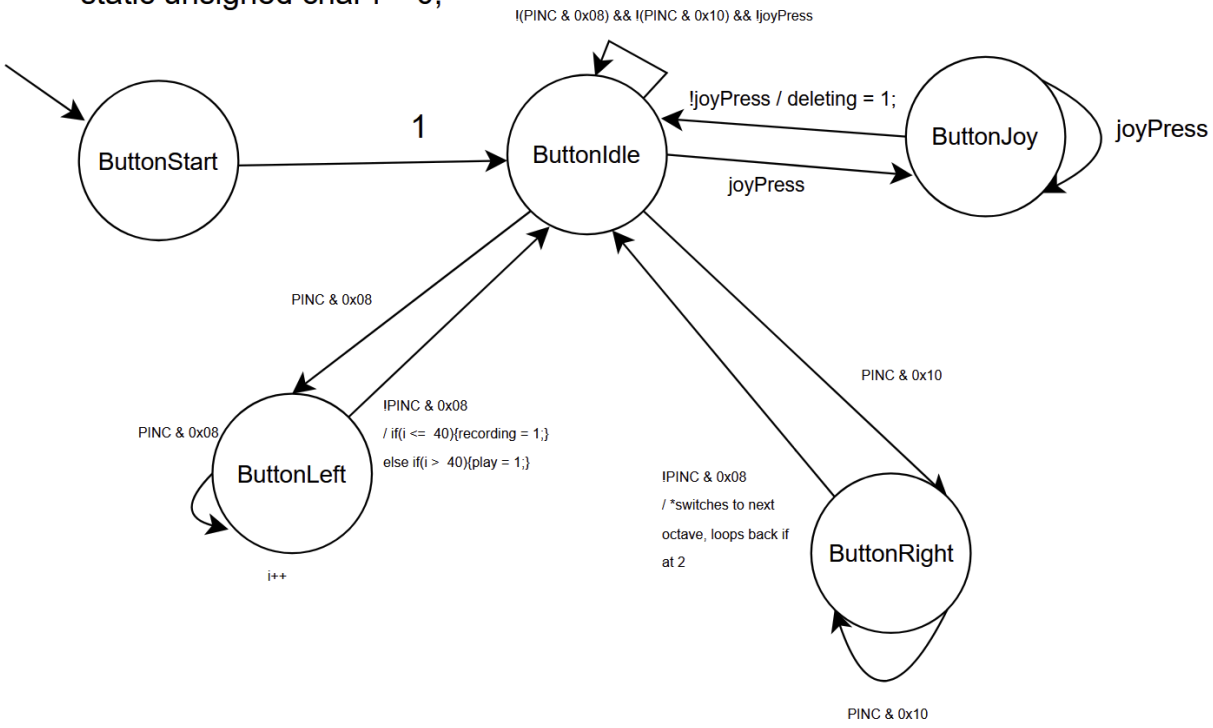



```
joyX = map(ADC_read(1), 0, 1023, 0, 100);
joyY = map(ADC_read(0), 0, 1023, 0, 100);
joyPress = (PINC & 0x04);
```

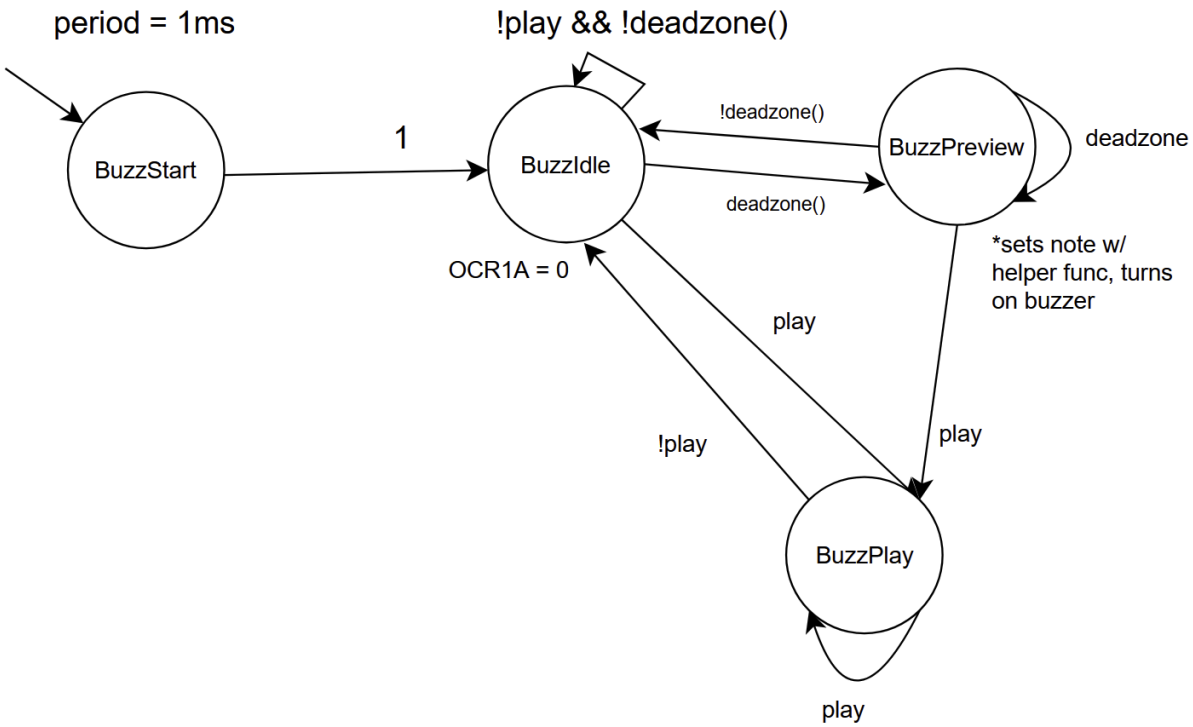
Buttons

period = 50ms

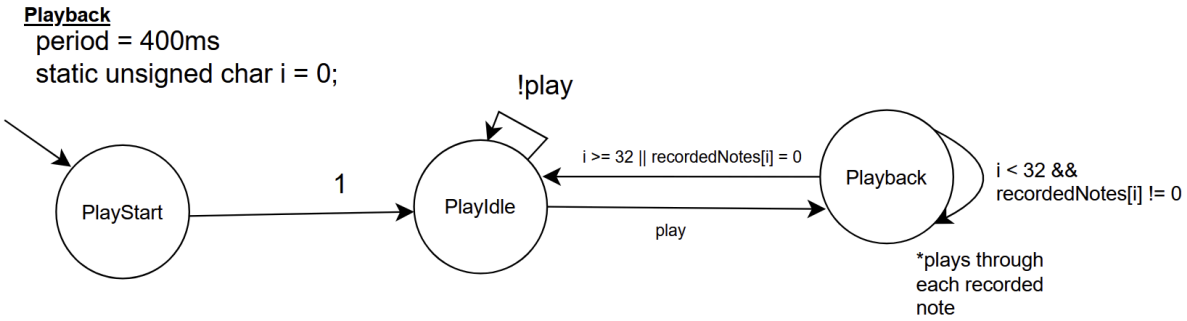
static unsigned char i = 0;



Preview



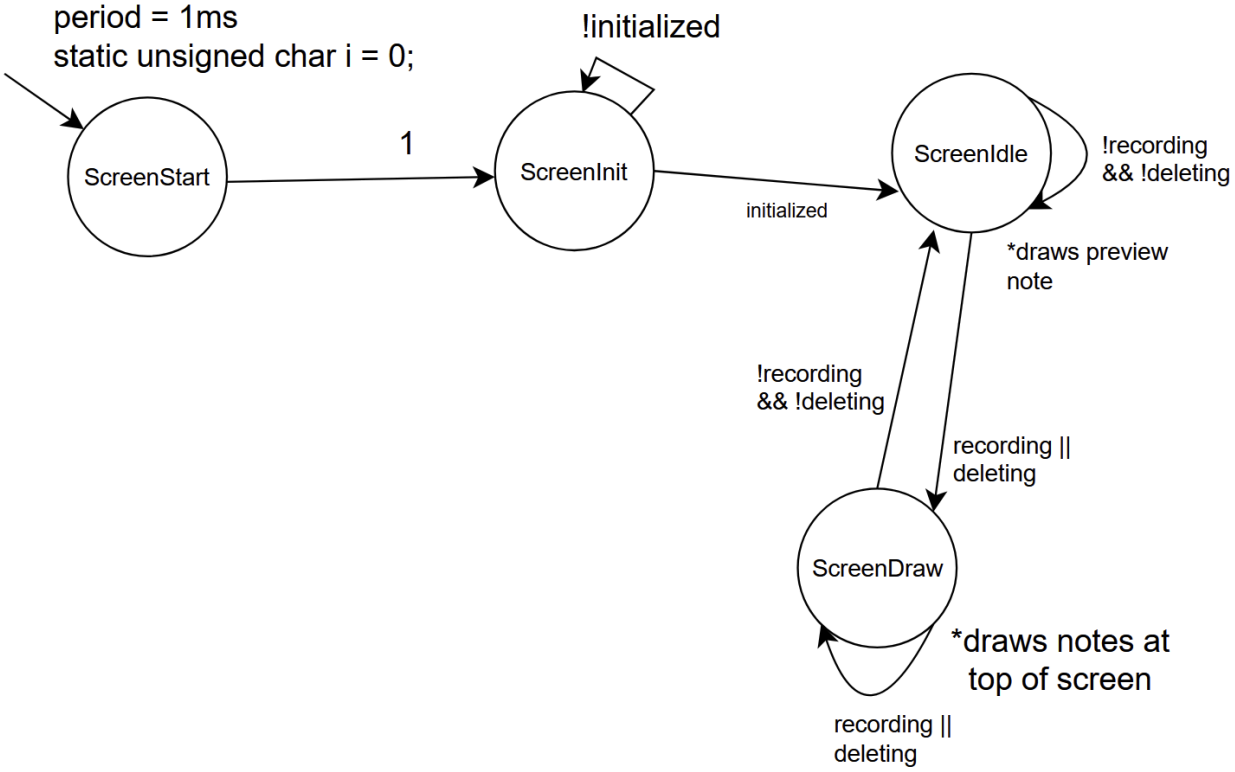
Playback



Screen

period = 1ms

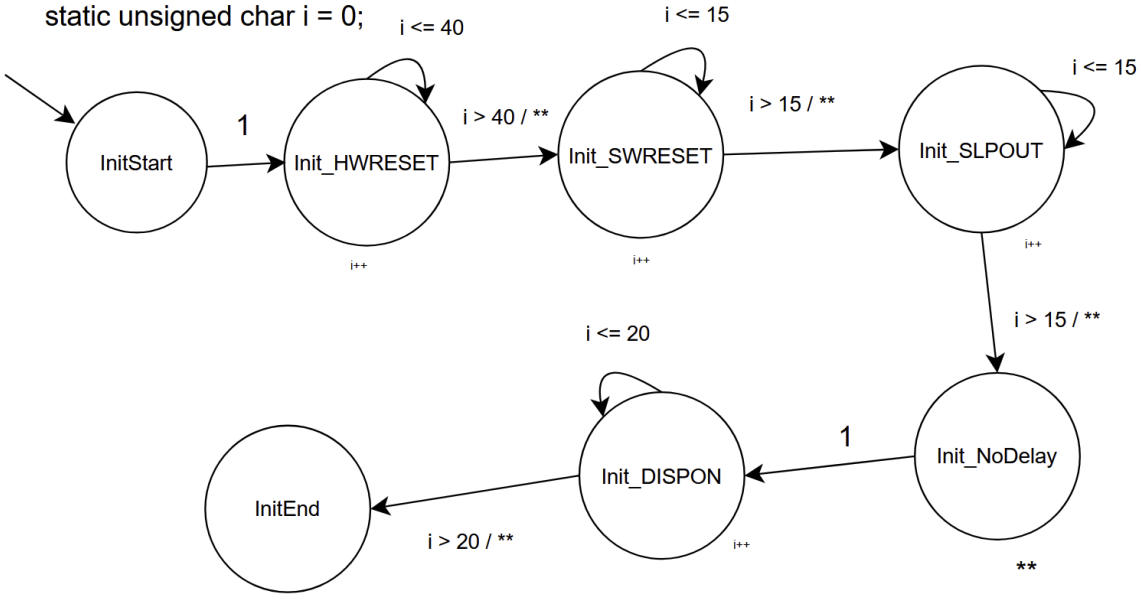
static unsigned char i = 0;



Initialization

period = 10ms

static unsigned char i = 0;

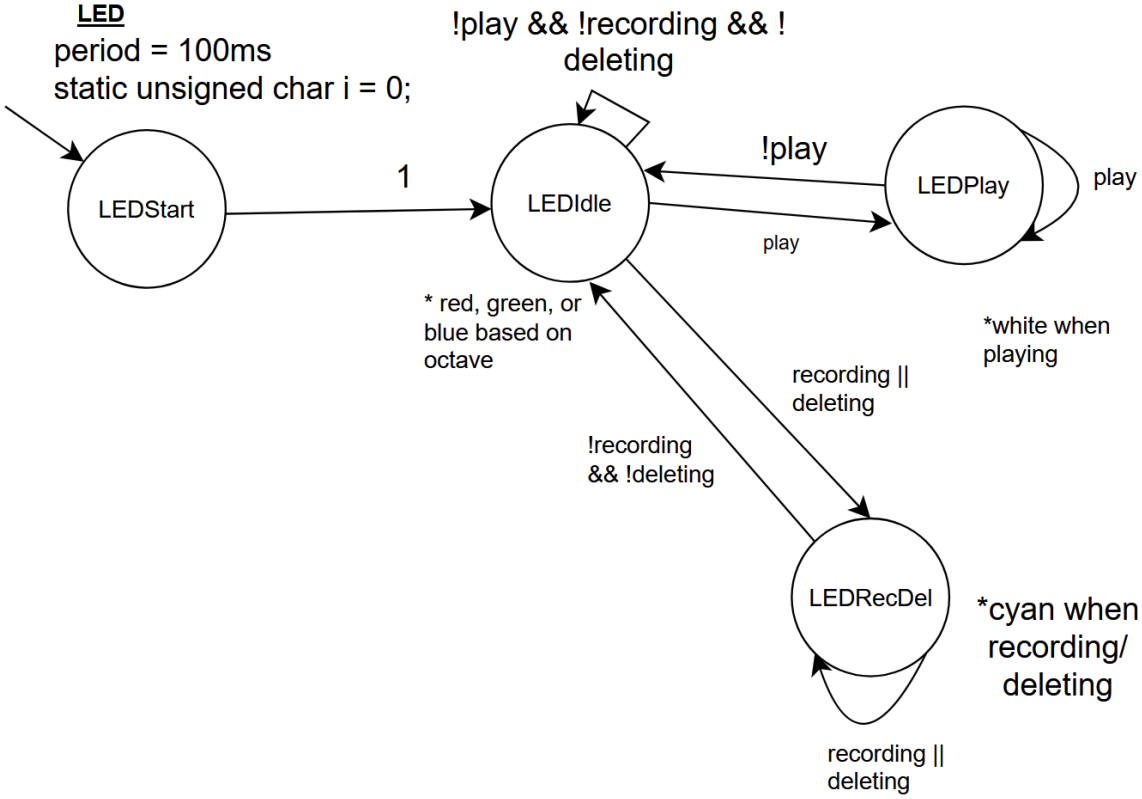


**sends commands/data
needed to initialize screen

LED

period = 100ms

static unsigned char i = 0;



Record

period = 1ms

static unsigned char noteRecorded = 0;

static int currentNote = 0;